

Q implement Queues inserting element at different Location.

Page No: _____

Date: _____



Queue using Stack.

push

pop

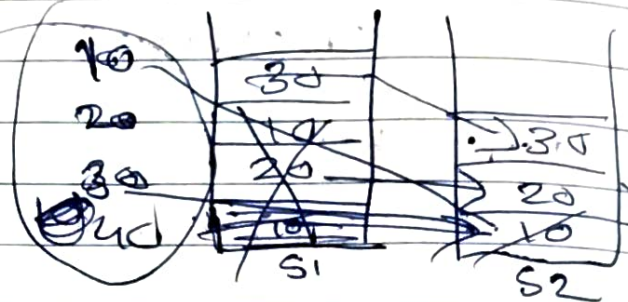
add element in Queue

~~1 2 3 4 5~~

10, 20, 30, 40, 50

if S1 empty add 10
add 20 but in
first 10 shift 20

S2



10
20
30
40

queue create using stack take two stack push element in stack shift the element into another stack again shift into old stack

element add first that remove first in stack

pop 1 remove 10, pop 2nd remove 20 in stack Last In first out

40
30
20
10

in queue

10
20
30
40

```
public static void add(int data)
```

```
{ if (!s1.isEmpty())
```

```
{ s1.push(s1.pop())
```

```
} s1.push(data);
```

```
import java.util.*;
```

```
public class Queue {
```

```
    static class Queue {
```

```
        static Stack<Integer> s1 =  
            new Stack<>();
```

```
        static Stack<Integer> s2 =  
            new Stack<>();
```

```
    public static boolean isEmpty() {  
        return s1.isEmpty();
```

```
    public static void add(int data) {
```

```
        while (!s1.isEmpty()) {
```

```
            helper s1.push(s1.pop());
```

```
        s1.push(data);
```

```
    while (!helper s2.isEmpty()) {
```

```
        s1.push(s2.pop());
```

```
    public static int remove() {  
        if (isEmpty()) {
```

```
            System.out.println("empty  
            queue");  
            return -1;
```

```

    }
    return sl.pop ();
}

```

```

public static int peek () {
    if (isEmpty ()) {
        System.out.println("empty stack");
        return -1;
    }
    return sl.peek ();
}

```

```

public static void main (String[] args)

```

```

{
    Queue<Integer> q = new ArrayDeque<>();
    Queue q = new Queue<>(); // use queue class
    q.add(10);
    q.add(20);
    q.add(30);
    q.add(40);
}

```

```

while (!q.isEmpty ()) {

```

```

    System.out.println(q.peek());
    q.remove();
}

```

```

}
}

```

Implement Queue using Stack

~~Sc = 0(n)~~

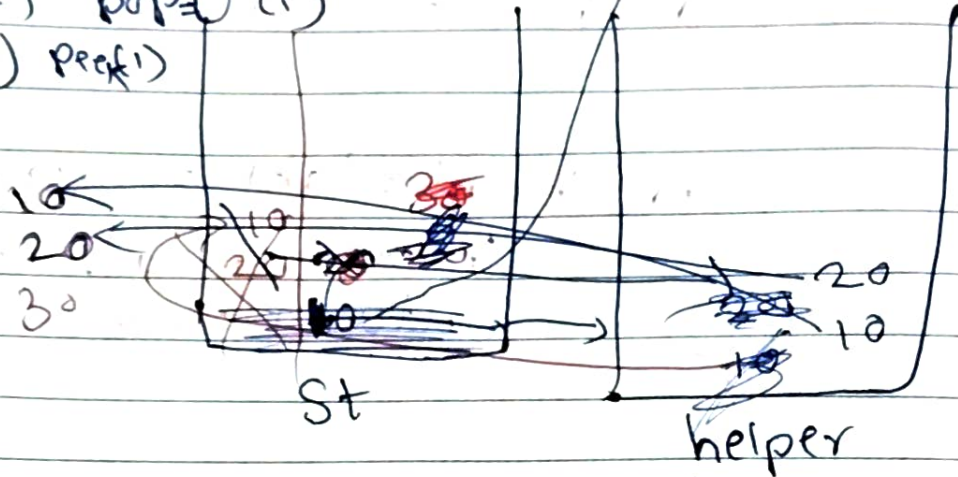
~~push(0(n))~~ Push(0(n))

~~pop(0(n))~~ pop(0(n))

~~peek(0(n))~~ peek()

10, 20, 30, 40

dry run



St > 0 push 10, bt push 20 after 10 So
10 shift in helper if (Stack.size() == 0)
helper.push(st.pop()) stack.push(x)

St empty if stack empty push a
push Stack not empty So shift in
helper. then again

10, 20 remove from St,

* helper.pop() This gives FIFO reverse
the order

I transfer elements only when helper is
empty if helper already has element
I direct pop from helper

* peek return front element without
removing it

move all element from Stk to helper
helper.push(stk.pop());

reverses the order, oldest element
comes to top of helper, Then

class MyQueue

Stack<Integer> stk;
Stack<Integer> helper;

public MyQueue() {
 stk = new Stack<>();
 helper = new Stack<>();
}

public void push(int x) {
 stk.push(x);
}

public int pop() {
 if (helper.empty()) {
 while (!stk.empty()) {
 helper.push(stk.pop());
 }
 }
 return helper.pop();
}

public int peek() {
 if (helper.empty()) {
 while (!stk.empty()) {
 helper.push(stk.pop());
 }
 }
 return helper.peek();
}

public boolean empty () {

return stk.empty () &&

helper.empty ();

}